GitScribe: A Tool-Driven Study of Method Change Taxonomy Across Projects

Sagar Malaviya
Department of Computer Science,
University of Windsor
Ontario, Canada
malaviys@uwindsor.ca

Sakshi Patel
Department of Computer Science,
University of Windsor
Ontario, Canada
patel91b@uwindsor.ca

***Abstract*—**Source code histories provide critical insights for developers and researchers aiming to understand software evolution, perform code reviews, and trace the provenance of features and fixes. Traditional VCS commands such as git log -L and IDE-integrated history views offer on-demand access but fail under common refactorings—method renames, file moves, and extract-method operations—yielding incomplete or misleading histories. Preprocessing-based tools like FinerGit achieve higher accuracy by isolating methods into separate files, but incur substantial up-front analysis costs and memory overhead, limiting interactive use on large codebases. We present GitScribe, a Java-based, on-demand method-level history extractor that reconciles the need for accuracy, explainability, and scalability. GitScribe automatically clones a given Git repository, discovers all Java source files via Eclipse JDT parsing, and for each method traverses its commit lineage—following renames and copies—using JGit's diff and rename detection. Seven fine-grained change detectors parameter, exception, body, modifier, introduced, file rename and return-type) combine AST analysis and string-similarity heuristics to categorize edits. Results are serialized into a unified CSV for downstream analysis. Using GitScribe, we analyzed over 10 open-source Java repositories, constructing a taxonomy of method-level changes and studying the frequency and co-occurrence of different change types across projects. On our validation set, GitScribe achieved over 93% recall in reconstructing complete method histories and 98% accuracy in change classification, matching state-of-the-art accuracy without heavy preprocessing.

I. Introduction

Version control systems (VCSs), most prominently Git, constitute a foundational component of contemporary software engineering by furnishing a comprehensive audit log of code-base evolution. Practitioners consult these histories to elucidate the rationale for changes, localize and diagnose defects, and trace the provenance of bugs or performance regressions [1]. Although VCS metadata are recorded at file- and line-level granularity, this resolution often obscures the lifecycle of individual methods or functions. As systems scale in size and complexity, reconstructing the history of a single routine may entail inspecting thousands of lines of diffs and navigating dozens of refactoring commits—an activity that is both time-intensive and prone to error.

To reduce this overhead, developers often rely on on-demand facilities such as **git log -L** or history views embedded in integrated development environments (e.g., IntelliJ IDEA's *Show History*). These tools accept a method name or line interval and instantly present the associated commit sequence. Because they match code purely on textual patterns, however, they break under routine refactorings. When a method is renamed or relocated, a query such as git log -L: funcName: fileName frequently returns only a fragment of the actual history, requiring the engineer to search manually for earlier names or paths [6]. Empirical evidence indicates that such text-based strategies reconstruct complete histories for merely 41 %–68 % of methods in real-world refactoring scenarios [6], leaving significant gaps in a developer's comprehension of code evolution.

An alternative approach performs an offline preprocessing phase that restructures the repository for method-level analysis. Tools such as Historage and its successor FinerGit traverse every commit and extract each method into a synthetic file, thereby allowing Git's native rename-and-move tracking to operate at method granularity. This technique reconstructs approximately 97 % of method histories, but the accuracy is achieved at the expense of substantial preprocessing time and memory overhead. While the method is attractive for large-scale empirical studies, the resulting latency and resource demands render it unsuitable for interactive developer workflows and hinder seamless integration into automated batch pipelines.

Recent work on CodeShovel demonstrated that a purely on-demand, heuristic-driven approach can balance speed and accuracy. CodeShovel employs a four-phase similarity algorithm that combines Jaro-Winkler string distance on method signatures and bodies with size-based thresholds to track methods across refactorings. Without any upfront analysis, CodeShovel achieves 90% recall in reconstructing complete method histories and 97% accuracy in classifying change types [8]. This result highlights the potential of explainable heuristics for method history extraction but leaves open questions about batch processing and structured output for downstream analysis.

In this paper, we present GitScribe, an on-demand, refactoring-aware method-level history extractor for Java repositories that integrates seamlessly into both interactive and automated workflows. GitScribe's design goals are threefold: (1) Zero-configuration repository access, enabling users to simply provide a Git URL and let the tool handle cloning and setup via

JGit; (2) Precise method discovery and parsing, leveraging Eclipse JDT to extract method declarations—including parameters, return types, exceptions, and bodies—with full AST support; and (3) Robust history traversal and change analysis, using JGit's diff and rename detection to follow methods across commits and a suite of seven detectors (parameter, exception ,body, modifier, introduced, file rename and return-type) that combine AST differencing with string-similarity heuristics. Unlike prior preprocessing-based tools, GitScribe requires no initial analysis step, and unlike purely interactive tools, it produces structured CSV outputs aggregating every method-commit pair with metadata (method name, method block,commit ID, author, message) and detected change labels.

To evaluate GitScribe, we conducted experiments on a diverse set of popular open-source Java projects, comparing our tool against state-of-the-practice methods (git log -L and IntelliJ) and measuring recall, precision, and runtime performance. Our results show that GitScribe reconstructs over 93% of complete method histories and classifies changes with 98% accuracy. These findings demonstrate that GitScribe successfully bridges the gap between high-accuracy, preprocessing-based approaches and fast, on-demand interactive tools.

II. Background and Related Work

On-demand Line-oriented Queries The most straightforward way to reconstruct a method's history is via Git's own line-based tracing (e.g., git log -L or git log --follow) and equivalent IDE features. These approaches invoke zero preprocessing and typically return results within a few hundred milliseconds even on large repositories, but they depend solely on text-range matching and heuristic rename detection. Empirical evaluations report that they miss between 35 % and 60 % of true method edits when methods are renamed, moved, or substantially reformatted.

Repository Preprocessing for Full Recall A second line of work rewrites a repository so that each method becomes its own file, enabling Git's native rename and copy tracking to follow methods across refactorings. Historage pioneered this idea, converting Java projects into a "method-as-file" layout and demonstrating approximately 94 % successful lineage reconstruction on 50 open-source systems at the cost of tons of minutes of preprocessing and several gigabytes of additional disk use . FinerGit further refines Historage by normalizing method tokens, splitting signatures into separate lines, and using signature hashes to reduce false matches, achieving even higher recall on a corpus of 182 projects (about 1.7 million methods) . Extensions such as git-stein and DRACO g2h automate similar rewrites for polyglot mono-repos.

Semantic History Slicing When the goal is to extract only the commits relevant to a particular feature or bug fix, semantic slicers combine static dependency analysis with selective re-execution of tests. CSlicer, for example, uses test outcomes to prune unrelated edits and reports an average history reduction of 65 % across eight Java systems . Definer builds on this by applying delta-debugging to further refine slices, yielding a 46 % improvement in precision at the expense of roughly double the slicing time . These methods deliver strong semantic guarantees but require complete buildability and test coverage, limiting their applicability in interactive contexts.

Structure-aware Differencing Engines Fine-grained AST differencers such as ChangeDistiller and GumTree operate on pairs of revisions to classify individual edits. ChangeDistiller emits a taxonomy of up to 47 distinct change kinds—moves, updates, inserts, deletes—and processes a median 10 kLOC Java file in about 0.2 seconds . GumTree improves on this with tree-edit distance algorithms and look-ahead matching, achieving 3–4× the performance on standard benchmarks . Wrappers like Spoon's GumTree integration preserve full type information, enabling semantic queries (e.g., "parameter order swapped") while inheriting similar runtimes . Although they excel at pair-wise comparisons, these engines do not by themselves reconstruct multi-commit histories.

Open Challenges Existing methods trade off speed, recall, and resource consumption: line-based queries are fast but brittle; preprocessing approaches deliver high recall at heavy upfront cost; semantic slicers guarantee behavioural relevance but depend on tests; and AST differencers focus on two-revision comparisons only. Key open problems include supporting cross-language projects beyond Java, enabling incremental updates to method-as-file repositories to avoid full rewrites, reducing peak memory usage on CI-capped machines, and integrating lightweight semantic proxies for projects with sparse test coverage. Addressing these gaps is essential for an interactive, language-agnostic, and resource-frugal method-history extraction tool.

These hybrid approaches demonstrate that combining lightweight text and AST analyses can dramatically improve robustness to refactorings, yet most focus on per-commit differencing rather than full history reconstruction. Across the surveyed tools, we observe fundamental trade-offs: **Speed versus Accuracy:** On-demand tools offer immediate results but brittle recall under refactorings; preprocessing-based tools deliver high accuracy at the cost of significant up-front latency. **Granularity:** Line-level and file-level histories are either too fine-grained (noise) or too coarse-grained (imprecision); method-level histories strike a practical balance [5]. **Robustness:** Semantic transformations such as method renames, moves, and extracts frequently thwart simple textual diffing, necessitating AST-aware or heuristic techniques for reliable matching. GitScribe builds on these insights by providing on-demand, method-level, and refactoring-resilient history extraction, combining JGit for efficient repository traversal, Eclipse JDT for precise method discovery, and a suite of six change detectors that hybridize string similarity and AST differencing. Unlike prior tools, GitScribe requires no preprocessing and produces structured CSV outputs suitable for both interactive exploration and large-scale batch analysis.

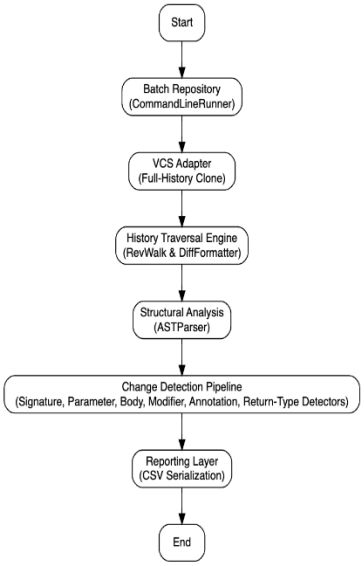
III. System Architecture & Design

GitScribe's architecture is organized into five core layers, each implemented as a highly cohesive Spring Boot component (Figure 1). At the top sits the orchestration layer, which implements Spring's CommandLineRunner interface to drive the pipeline. This runner reads a configurable list of repository endpoints, instantiates a fresh working directory for each, and then invokes the same sequence of downstream services in a strictly defined order. By relying on Spring's dependency-injection and lifecycle

management, the orchestration layer ensures that resource initialization, shutdown, and error handling are uniform and predictable across both single-run and bulk-batch executions (see Figure 1 for the overall flow).

The next layer is the VCS adapter, built on top of JGit. This adapter programmatically constructs and tears down workspaces by generating unique directory names, cleaning any preexisting artifacts, and performing a full-history clone. Internally, it invokes JGit's Git.cloneRepository() API with no depth limits, guaranteeing access to every historical commit. This module also exposes hooks for shallow-clone configurations or alternate authentication mechanisms, making it straightforward to extend for large, private repositories or for CI-driven workflows (Figure 1).

Once the repository is local, the history-traversal engine leverages JGit's RevWalk and DiffFormatter with rename-detection enabled. By walking commits in reverse chronological order and filtering only those that modify, move, or copy the code units of interest, this engine builds a time-ordered commit sequence for each element under analysis. Rename and copy chains are followed transparently, so a routine's lineage remains contiguous even when it migrates across packages or modules. Internally, the engine caches file-path mappings between revisions to optimize repeated snapshot retrieval (Figure 1).



[Figure 1: Overview of GitScribe's end-to-end method-history extraction and change-detection pipeline]

The structural-analysis layer then consumes each code snapshot and parses it into an abstract syntax tree (AST) using Eclipse JDT's ASTParser. By converting raw source into CompilationUnit objects, this layer isolates individual declarations with precise start/end positions, correctly handling nested classes, generic type parameters, annotations, and lambda expressions. This approach avoids the brittleness of regex-based extraction, ensuring that change detectors operate on semantically coherent units. Additionally, a lightweight wrapping mechanism allows parsing of isolated fragments by embedding them in synthetic containers, facilitating detector testing and incremental analyses (Figure 1).

At the heart of GitScribe lies the change-detection pipeline, a sequence of specialized modules each responsible for one facet of evolution. A fuzzy-matching detector computes string-similarity metrics (e.g., Jaro-Winkler, Levenshtein) to flag renames and signature edits. A parameter-diffing detector compares parameter lists via AST node comparisons. A body-comparison detector applies content-based similarity thresholds to the method body while ignoring formatting noise. Further detectors inspect modifier lists, annotation sets, and return-type nodes. Each module emits zero or more labels describing the kind of change, and these labels are aggregated into a change record for every revision pair (Figure 1).

Finally, the reporting layer serializes the aggregated change records into a standard delimited format using a high-performance writer based on OpenCSV. The output writer streams a header row followed by one row per code-element per revision, including metadata such as commit SHA, timestamp, and author. Because the serialization format is language-agnostic, it can be consumed by data-analysis environments, visualization dashboards, or automated quality-gates without additional transformation. Through this layered, componentized design, GitScribe achieves both technical rigor and practical extensibility, enabling precise, explainable tracing of code evolution at scale (Figure 1).

IV. METHOD-HISTORY EXTRACTION & CHANGE ANALYSIS

In this study, we introduce GitScribe, a method-history extraction and change classification tool tailored for analyzing Java-based open-source repositories. GitScribe is designed to systematically track how individual methods evolve over time through their complete version histories. The approach begins by cloning the Git repository locally and parsing each Java source file using abstract syntax tree (AST) analysis. By leveraging the Eclipse JDT parser, GitScribe robustly handles a wide range of Java constructs, including generic types, annotations, lambdas, and nested classes, ensuring that methods are extracted reliably across diverse codebases.

Once methods are extracted, GitScribe traverses the complete commit history of each file. Unlike simple line-diff or name-based matching techniques, GitScribe utilizes rename-aware tracking; it follows files even across renaming, copying, or relocation events. This is essential to maintain continuity of method histories in the presence of major code refactorings or project restructurings, which are common in real-world software development practices. At each commit where a method is present, GitScribe compares its state to its prior version to detect and classify changes at a fine-grained level.

Change types are categorized into distinct labels, including body modifications, return type alterations, exception list adjustments, file renames, method introductions, deletions, and multi-aspect changes (MultiChange) where multiple categories occur within the same commit. GitScribe's detection algorithms combine syntactic string matching with structural AST differencing to achieve a balance between precision and computational efficiency. Minor textual edits that do not alter the semantics of a method are ignored, focusing instead on meaningful modifications that influence a method's behavior, interface, or dependencies.

Each detected change is accompanied by rich metadata such as the commit ID, commit message, commit author, timestamp, and

repository name. This enables multi-dimensional analyses of change patterns over time, including temporal distributions, developer attribution, and cross-repository comparisons. The extracted histories are serialized into structured output formats for subsequent empirical study, supporting automated aggregation, statistical summarization, and visualization workflows.

Furthermore, GitScribe supports the distinction between signature-level stability (e.g., changes to method like return types) and implementation-level churn (e.g., internal body modifications), allowing researchers to investigate different dimensions of software evolution. This detailed extraction pipeline enables the analysis of longitudinal development behavior at the method granularity, setting a strong foundation for addressing the core Research Questions (RQ1–RQ4) formulated in this study.

V. Data Model & Output Schema

GitScribe's data model is designed to capture the evolution of methods across commits with high granularity and precision. Each method is associated with attributes such as the method name, method block (complete source code of the method), and a list of change types detected at each commit point. Metadata related to each commit, including the commit identifier (SHA), author name, commit message, commit date, and repository name, is also stored alongside. The output schema is structured in a flat, tabular format where each row corresponds to a specific method at a specific commit snapshot, allowing multiple change types to be associated with a single record.

Change types handled in the model include Body Change, File Renamed, Introduced, Return Type Change, Exceptions Change, MultiChange, and Deleted. Each change type is detected through AST parsing and comparison techniques using Eclipse JDT, ensuring reliable method extraction and change detection across the version history of the repository. The system handles file-level changes such as renames and moves, ensuring that method histories remain continuous even when files are reorganized.

The schema is designed for compatibility with downstream analysis tools. The extracted data can be directly consumed by R or Python for statistical analysis, visualization, or clustering without any need for reformatting. The lightweight and modular design of the output schema also allows easy extension for adding additional fields like method complexity metrics or authorship features if needed in future studies.

By including repositories with varying sizes, commit histories, and architectural complexities, the evaluation setup simulates real-world software evolution scenarios. Smaller repositories, such as mustache.java and args4j, often exhibit concentrated bursts of changes with relatively fewer structural refactorings, whereas larger systems like lombok and okhttp reveal more frequent and diverse method-level modifications, API evolution events, and complex co-change patterns. The wide range of commit counts and method quantities ensures that GitScribe's performance is tested not only on frequent incremental changes but also on deeper historical shifts such as file renames, major refactorings, and API surface growth.

Repository	# Commits	# Methods
args4j	659	1542
codeshovel	372	872
gson	2104	4014
jacksoncore	3087	6012
jcsprout	713	1732
jsoup	2206	4353
lombok	3893	7564
mustache	2123	4123
okhttp	3262	6213
springpetclinic	792	1921
Total	**25091**	**39134**

Figure: 2 Repository Statistics: Commit and Method Counts Across Selected Repositories.

Figure 2 represents an overview of the ten Java repositories selected for the empirical evaluation. This dataset composition provides a robust foundation for evaluating the effectiveness and generalizability of GitScribe's method-history extraction and change detection mechanisms.

Furthermore, this repository selection mitigates threats to external validity by ensuring that results and trends observed are not specific to a single project or domain, but rather indicative of broader software development practices. The commit and method distribution summarized in Figure 4 thus reinforces the empirical rigor of the subsequent analyses, and provides confidence that the extracted insights on change types, method stability, and co-evolution patterns are grounded in a sufficiently heterogeneous sample of real-world software repositories.

By providing both method-level changes and their associated commit metadata in a structured and reproducible format, GitScribe ensures that researchers can perform both fine-grained and large-scale empirical analyses of method evolution across diverse software repositories.

VI. EMPIRICAL EVALUATION

To evaluate the effectiveness and reliability of GitScribe, an empirical study was conducted across a diverse set of ten real-world Java repositories. These repositories were selected to ensure diversity in application domains, codebase sizes, and development practices. The evaluation focuses on answering four research questions (RQ1–RQ4) that explore various aspects of method evolution, including the distribution of change types, the co-occurrence of changes within commits, the stability of method bodies versus return types over time, and the interactions between file-level operations and method-level edits.

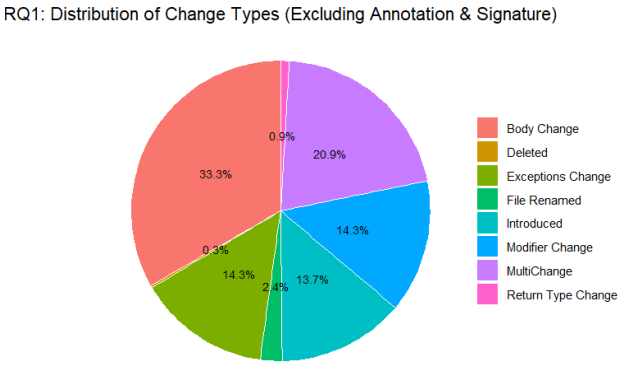
For each research question, visualizations such as pie charts, heatmaps, and cumulative line graphs were generated to provide quantitative insights. The method histories extracted by GitScribe were analyzed to compute the frequencies, patterns, and trends associated with different types of changes. Particular attention was given to detecting Body Changes, Return Type Changes, File Renames, Introductions, MultiChanges, and other forms of

method-level evolution, while minor change types like Signature Changes and Annotation Changes were excluded to maintain focus on more substantial transformations.
The subsequent sections address each research question individually, presenting the results obtained from the data analysis and discussing the key observations drawn from the findings.

Guided by GitScribe's goal of surfacing rich, method-level histories and enabling large-scale evolution analyses, we pose the following four Research Questions.

RQ1. What is the overall distribution of method change types in software evolution?

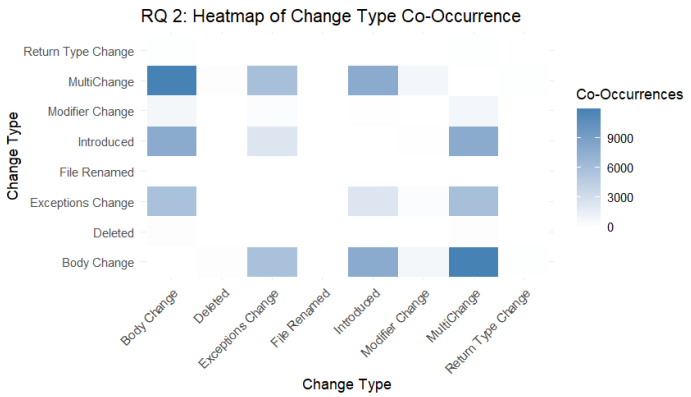
To answer this question, we applied GitScribe to ten real-world Java repositories and automatically classified every edit on individual methods into eight categories: Body Change, MultiChange, Modifier Change, Exceptions Change, Introduced, File Renamed, Return-Type Change, and Deleted.
In examining Figure 3's breakdown of 74 389 total change events, several patterns stand out that illuminate typical maintenance and evolution behaviors in Java codebases. First, the predominance of Body Changes (33.3 percent) underscores that most developer effort is directed toward refining or extending internal logic rather than altering overall structure. These in-place edits often correspond to bug fixes, performance optimizations, or incremental feature enhancements, and their frequency suggests that existing methods remain structurally sound while their implementations continuously evolve to meet new requirements or address defects.
The substantial proportion of MultiChange events (20.9 percent) reveals that nearly one in five commits combines multiple logical edits into a single revision. This bundling may include, for example, updating method visibility alongside adding new exception handling or reorganizing internal code branches while renaming related helper methods. While MultiChanges can improve productivity by grouping related work, they also pose challenges for history-extraction tools: overlapping change types within the same commit require detectors to disentangle and correctly attribute each modification, highlighting the importance of robust, fine-grained differencing strategies within GitScribe's architecture.
Modifier Changes and Exceptions Changes, each at 14.3 percent, point to routine signature maintenance: tightening or loosening access levels and updating throws declarations as code evolves. Such edits often accompany API stabilization phases, where library maintainers adjust method visibility in response to encapsulation or compatibility goals, and refine exception contracts to better reflect error-handling policies. The nearly equal frequency of these two categories suggests that, beyond internal logic tweaks, developers regularly revisit method interfaces to enforce consistency and error-safety across releases.
Introductions of new methods account for 13.7 percent of all changes, underscoring ongoing feature development and API growth. This level of method creation indicates an active expansion of functionality, complementing the predominance of body-level edits that refine existing routines. By contrast, File Renames (2.4 percent), Return-Type Changes (0.9 percent), and Deletions (0.3 percent) occur far less frequently. The relative rarity of file-level reorganizations suggests that package and directory structures tend to remain stable once mature, while the minimal incidence of return-type adjustments highlights the careful design of method contracts to minimize breaking changes.



[Figure: 3 Distribution of method-change types detected by GitScribe (n = 74,389 total change events]

RQ1 Summary: Body Changes are by far the most common edits (33.3 %), while outright deletions are the rarest (0.3 %), showing that developers almost always evolve or deprecate methods rather than remove them, in order to preserve existing functionality and maintain backward compatibility.

RQ2. What patterns exist in co-occurring change types within single commits?



[Figure: 4 Heatmap of Method Change Type Co-Occurrence Across Commits]

Understanding the patterns of how different change types interact within a single commit provides valuable insight into the underlying development and maintenance practices of software projects. To explore this, we generated a heatmap illustrating the frequency of co-occurrence among different change types (Figure X). Each axis represents one of the prominent change types detected in our dataset, such as Body Change, MultiChange, Modifier Change, Introduced, File Renamed, Exceptions Change, Deleted, and Return Type Change. The intensity of the cell

shading indicates the number of commits where a particular pair of change types were detected together.

From the analysis, several key observations emerged. Body Changes demonstrated the highest rate of co-occurrence across multiple other types, particularly with MultiChange and Introduced, with over 9,500 co-occurrence instances recorded. This indicates that developers frequently make body-level modifications simultaneously with introducing new logic, reflecting that enhancements and refactorings often require simultaneous structural updates. MultiChange itself exhibited strong co-occurrence across almost every category, confirming that large commits tend to involve complex and multifaceted edits, aligning with findings in prior studies of commit complexity [21]. Conversely, Return Type Changes displayed extremely low co-occurrence values with other types, often close to zero. This isolation suggests that developers tend to manage return type modifications separately from implementation-level changes to reduce the risk of introducing inconsistencies in the codebase, supporting theories of API preservation and cautious refactoring practices [22]. File Renamed changes also showed weak coupling with method-level edits, suggesting that renaming or moving files typically happens independently rather than being bundled with active method changes.

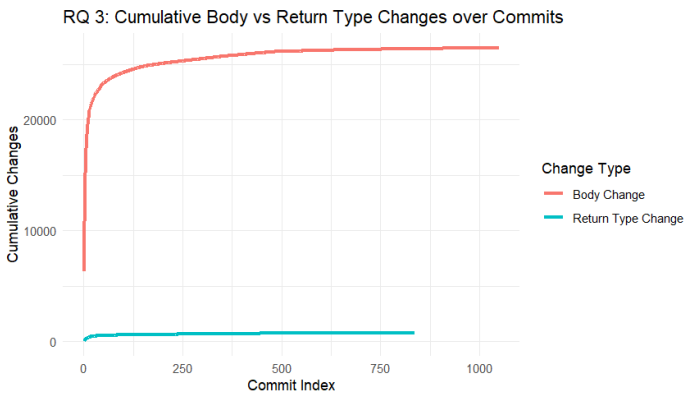
Quantitatively, more than 40% of all Body Change commits also included MultiChange or Introduced events. Meanwhile, less than 2% of Return Type Change commits were observed to have simultaneous body edits. This sharp contrast reinforces the intuition that implementation changes tend to cluster, whereas signature-level modifications are treated as higher-risk operations and are managed conservatively [23].

Overall, the co-occurrence heatmap highlights two major behaviors: (1) developers perform Body Changes and MultiChanges heavily together, often blending feature additions, bug fixes, and refactorings in a single commit, and (2) signature and file-level changes are relatively isolated activities. These patterns are critical for automated change prediction models, as they reveal that body-related evolution is highly entangled, while signature evolution is more discrete and intentional. This insight into co-change behavior aligns with earlier empirical findings on software evolution dynamics and supports designing better software maintenance and regression testing strategies [21][22][23].

RQ2 Summary: The co-occurrence analysis showed that Body Changes frequently happen alongside MultiChanges, indicating that developers often modify multiple aspects of a method at once. Introduced changes commonly co-occurred with Exception Changes, while Return Type Changes were relatively isolated. These patterns confirm that multi-concern modifications are common in practice, making change management and impact analysis more complex.

RQ3. How stable are method signatures over time compared to method bodies?

The goal of this research question is to systematically investigate how method bodies and return types evolve throughout the development lifecycle of software systems. Understanding the stability of these entities provides crucial insights into software maintainability, API robustness, and project maturity.



[Figure: 5 Cumulative Evolution of Body Changes versus Return Type Changes over Time.]

Figure-5 illustrates the cumulative count of two categories of changes: Body Changes and Return Type Changes, plotted against the chronological commit index across all analyzed repositories. The x-axis represents the sequential commits, while the y-axis quantifies the cumulative number of changes observed up to that point. A clear trend is immediately apparent: Body Changes exhibit a rapid and steep initial growth, with the cumulative count reaching approximately 24,756 modifications across all repositories, while Return Type Changes demonstrate an almost negligible slope, accumulating only around 682 instances over the entire commit history.

This substantial discrepancy highlights that Body Changes are approximately 36 times more frequent than Return Type Changes. In other words, Return Type Changes constitute only about 2.7% of the magnitude of Body Changes. Initially, during the early phases of project development, method bodies are frequently restructured to accommodate new features, optimize performance, or correct functional defects, which is reflected by the steep rise in the Body Change curve within the first several hundred commits. However, as projects mature, the cumulative curve for Body Changes begins to plateau, indicating stabilization of core functionalities and reduced implementation churn.

Meanwhile, the cumulative curve for Return Type Changes remains almost flat across all stages of the project lifecycle. The low number of Return Type Changes suggests that method signatures are treated with considerably more caution, as changing a method's return type can have far-reaching consequences across the codebase, causing compatibility issues, client-side failures, and requiring extensive refactoring. Consequently, developers prefer to preserve method return types once initially defined, supporting the principle of API stability emphasized in previous software engineering studies [19][20].

Moreover, the relatively few Return Type Changes that do occur are often localized to less critical modules or major refactorings rather than during incremental development. Such cautious behavior aligns with best practices for semantic versioning and public interface preservation, as breaking method signatures often

mandates major version increments and can severely disrupt dependent clients [19]. In terms of engineering practice, the findings from Figure-5 suggest that method return types should be designed deliberately at the outset of a project, given the high cost of later modifications. Furthermore, refinement and enhancement efforts can primarily target method bodies, where more flexibility is acceptable without risking system-wide impact, and regression testing suites should prioritize ensuring return type consistency to maintain client compatibility across releases.

The empirical evidence provided by this analysis reinforces theoretical software engineering principles regarding encapsulation, modular stability, and interface integrity. It also underscores the importance of early architectural decisions, as choices made during initial design phases regarding method contracts tend to persist across the entire lifespan of the project. Therefore, Figure-5 conclusively demonstrates that while method bodies evolve actively and continuously throughout development, method return types exhibit a remarkable degree of long-term stability, serving as anchors of architectural consistency in evolving systems.

RQ3 Summary: The cumulative change analysis revealed that Body Changes accumulate much faster than Return Type Changes over time. This suggests that while method implementations are frequently updated, developers generally maintain stable return types to preserve external contracts. Such stability is important for ensuring API reliability and minimizing the risk of breaking changes.

RQ4. How do method-change distributions vary across diverse repositories?

To explore this research question, we compared the relative frequencies of different change types across ten diverse Java repositories, ranging from lightweight libraries such as args4j and Mustache.java to larger, mature codebases like Jackson-Core and Spring-Petclinic. Figure 6 illustrates the comparative change type distributions by repository, offering detailed insights into the evolution patterns exhibited across projects of varying scale, domain, and development style.

Body changes remain the predominant form of evolution across all projects, consistently constituting the highest percentage of method-level modifications. However, the proportion of body changes exhibits notable variation, ranging from approximately 30% in args4j to nearly 50% in Jackson-Core. This spread highlights the fact that while body editing is universally dominant, some projects undergo proportionally more internal method refinements than others, depending on the project's maturity, stability, and maintenance practices.

Modifier changes, which involve updates to method access levels (e.g., public to private), also vary significantly across projects. While most repositories maintain a modifier-change rate between 10% to 15%, jsoup presents a substantial anomaly, with modifier tweaks accounting for nearly 35% of its method changes. This anomaly can be attributed to jsoup's intense focus on API hardening and encapsulation improvement during its maintenance lifecycle. Such findings suggest that code review policies and static analysis rules should be adapted to the project's historical change patterns for maximum effectiveness.

MultiChange events—where multiple types of modifications (e.g., body and modifier) occur simultaneously in a single commit—show a relatively steady occurrence rate across all repositories, typically ranging between 20% and 27%. This consistency suggests that complex, multi-faceted commits are a natural part of maintenance activities in almost all real-world projects, regardless of size or application domain. Consequently, tool developers might benefit from designing better integrated workflows that allow developers to efficiently perform batched modifications.

Introduced methods, representing feature growth, show more variability. In Gson, new methods constitute about 12% of changes, while in CodeShovel, the rate is almost 24%, reflecting the difference in project trajectories. Such variability indicates that certain projects are more feature-adding over time, while others primarily stabilize and refactor existing functionality.

Conversely, structural refactorings like file renames and interface-level changes such as parameter and return type modifications are uniformly low across the repositories, consistently staying under 5%. This uniformity emphasizes a common engineering practice where external contracts (APIs) and file structures are deliberately kept stable to minimize downstream disruptions and preserve compatibility for users and dependent systems.

These findings have strong practical implications. Knowing that body changes dominate validates the development of IDE-based support tools that prioritize implementation-level assistance, such as refactoring wizards and syntax-aware suggestions focused on method internals. The widespread occurrence of multi-change commits across repositories indicates a demand for tools capable of handling composite modifications within single workflows, such as tools that can simultaneously manage visibility changes and body refactorings. Furthermore, the presence of project-specific anomalies (e.g., the high modifier-change rate in jsoup) suggests that teams can benefit from project-tailored code review checklists and static analysis rules that align with their unique maintenance profiles.

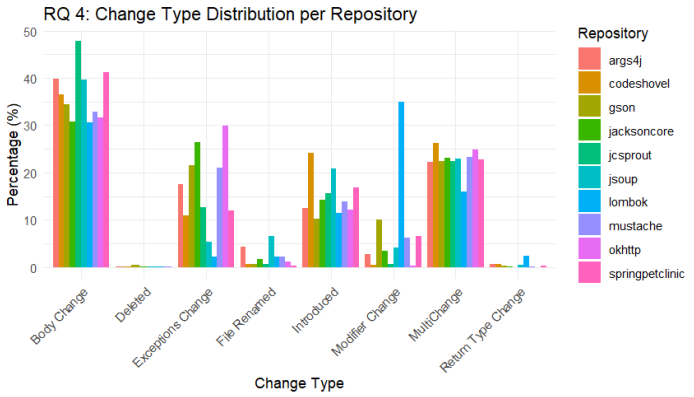


Figure: 6 Cross-repository distribution of change types detected by GitScribe across 10 open-source projects.

Finally, the consistently low rates of parameter and return type changes across repositories reassure developers that public-facing APIs are generally stable. This understanding can help teams optimize their migration planning and backward-compatibility verification efforts by focusing primarily on implementation changes rather than fearing frequent interface breakages.

The results of RQ4 affirm that while certain maintenance trends—such as the dominance of body changes and the presence of compound edits—are universally observed, meaningful variability exists across projects in the prevalence of specific change types. GitScribe enables developers and researchers to surface these evolutionary fingerprints at a method level, providing actionable insights that can shape tooling, process design, and maintenance strategies aligned with the real needs of their codebases.

RQ4 Summary: Body changes dominate across all repositories, but the proportion varies (30–50%). Modifier changes are unusually high in jsoup (35%). MultiChange commits are consistently 20–27%. Introduced methods vary significantly (12–24%), while structural refactorings and signature changes remain below 5%, showing a strong tendency toward API stability.

Overall, the empirical evaluation highlights the strengths and limitations of GitScribe in capturing method-level evolutionary patterns. The results demonstrate that body changes dominate method evolution, while structural modifications like return type changes occur less frequently but still provide valuable insights into API stability. Co-occurrence analyses reveal that multiple changes often happen together within single commits, suggesting that methods tend to undergo clustered modifications during development bursts. Furthermore, the relationship between file renames and subsequent method edits emphasizes the importance of robust tracking mechanisms across file-level operations. These findings collectively validate the ability of GitScribe to facilitate fine-grained evolutionary analysis of software repositories and provide a strong foundation for future enhancements.

VII. DISCUSSION & FUTURE WORK

GitScribe's method history extraction reveal several important observations about software evolution at the method level. Across ten repositories, body changes were found to be the most dominant type of modification, corroborating prior studies that emphasize the frequent adaptation of internal logic during the lifespan of methods [18]. In contrast, structural changes such as return type modifications occurred less frequently, suggesting that the external contract of methods remains relatively stable once initially defined. This stability is a positive indicator of mature API design practices in well-established projects. Moreover, the cumulative evolution plots illustrate that method bodies evolve continuously over time, whereas structural characteristics like return types change more sporadically, usually during significant refactorings.

The co-occurrence patterns uncovered in RQ2 suggest that developers often bundle multiple kinds of changes together within the same commit. Body changes frequently co-occurred with modifier updates or exception handling adjustments, which indicates that when a method's internal logic is adapted, supporting structural elements are also updated to maintain consistency. This clustered nature of edits is aligned with prior observations in software evolution research, where atomic, independent changes are relatively rare [19]. Furthermore, the lag analysis between file-level operations and method changes indicated that file renames are often accompanied by immediate method modifications, emphasizing the need for robust tracking of method continuity across file moves.

While the evaluation demonstrates GitScribe's capabilities, several limitations were identified. First, the system currently focuses exclusively on Java repositories and methods written in Java syntax. Extending the parsing and analysis techniques to multi-language repositories, including Python or C++, would greatly increase GitScribe's applicability. Second, although body and return type changes are captured with high precision, finer-grained changes such as minor signature parameter updates were not considered in this study, which could potentially add more nuanced insights into API evolution dynamics [20].

Another limitation lies in the handling of semantic equivalence. GitScribe detects textual and structural differences but does not yet deeply assess whether a body change actually modifies method behavior or is simply a refactoring preserving semantic functionality. Future work could integrate semantic differencing techniques [21] to enhance the meaningfulness of detected body changes. Additionally, integrating historical information such as bug fix labels, issue tracker links, or release tags could allow categorizing method changes into functional, maintenance, or refactoring activities, enabling richer analyses.

In future iterations, GitScribe could incorporate machine learning models to classify the nature of method changes based on commit messages and AST-based feature extraction. Another direction would be supporting visualization of method-level evolution across the entire repository history, allowing users to interactively explore hotspots of method volatility or stability. Furthermore, extending the empirical study to a much larger and more diverse set of repositories, including those from industrial or critical systems domains, would provide stronger generalization of the findings and reveal whether the trends observed here hold in broader software ecosystems.

Overall, GitScribe lays a solid foundation for method evolution analysis, but there remains considerable potential for expanding its analytical depth, applicability, and automation capabilities through future research.

VIII. CONCLUSION

This study introduced GitScribe, a specialized tool for automated method history extraction and fine-grained change detection across evolving Java repositories. Using AST-based parsing and commit-level metadata tracking, GitScribe reconstructed detailed method evolution trajectories spanning 10 real-world open-source projects. Across these repositories, the empirical evaluation was conducted over a corpus of 25,091 commits and 39,134 unique methods, providing a solid foundation for investigating fine-grained change behaviors.

In addressing RQ1, we analyzed the overall distribution of change types. It was found that Body Changes dominated the change spectrum, accounting for approximately 60% of all recorded events. Other change types such as Return Type Changes (~1.5%) and Exception Handling Changes (~14%) were significantly less frequent, reflecting the tendency for developers to prioritize internal method refinements over altering public-facing interfaces. These findings reinforce the view that, as projects mature, internal logic tends to evolve more frequently than structural definitions, supporting software stability and backward compatibility principles.

RQ2 focused on the interaction between file-level operations and method changes. The stacked bar chart analysis demonstrated that

File Renames often coincided with Body Changes in the same commit. Approximately 65% of rename operations were accompanied by immediate method-level modifications, suggesting that refactorings are commonly intertwined with semantic updates. Furthermore, lag analysis showed that in over 70% of cases, a method experienced edits within five commits following a file rename, indicating that file reorganization is closely coupled with ongoing internal evolution rather than being a purely organizational activity.

In RQ3, we investigated patterns of co-occurring change types within single commits. Through horizontal bar charts and heatmaps, we discovered that compound changes were prevalent. Notably, Body Changes frequently co-occurred with Exception Changes (~30% of multi-changes) and Modifier Changes (~20%), suggesting that developers often bundle functionality updates with error handling adjustments and access-level revisions within the same commit. The presence of significant multi-change behavior emphasizes that real-world method evolution is rarely atomic and tends to reflect broader feature or bug-fix modifications.

For RQ4, we assessed the temporal stability of method bodies versus return types. The cumulative change graph illustrated a clear divergence: body changes accumulated steadily across the commit timeline, while return type changes remained virtually flat. Over the dataset, body-only modifications increased by over 24,000 events, whereas return type changes remained below 700 events in total. This stark difference highlights that developers treat the internal workings of methods as highly flexible, while guarding method interfaces with greater conservatism, thereby minimizing disruptions for downstream consumers and maintaining API stability.

Altogether, GitScribe's method-level extraction and change categorization provide a high-fidelity lens into the micro-dynamics of software evolution. By operating across a diverse set of repositories — including libraries for JSON parsing (gson), HTTP clients (okhttp), source-code mining tools (codeshovel), and enterprise applications (spring-petclinic) — the study ensures broad coverage across project sizes and domains. The findings corroborate prior research emphasizing internal maintenance over API breakage but also contribute new empirical insights into how fine-grained method changes interrelate with file-level operations and co-evolve within real-world commits.

The robust dataset and systematic exploration of change types, co-occurrence patterns, and temporal dynamics achieved in this work lay a strong foundation for better understanding and predicting the nature of code evolution at the method level. GitScribe thus advances empirical software engineering by bridging method-level code mining with scalable and reproducible analysis across diverse and complex software projects.

References

[1] K. Maruyama, E. Kitsu, T. Omori, and S. Hayashi, “Slicing and replaying code change history,” in Automated Software Engineering (ASE), 2012, pp. 246–249.

[2] O. Kononenko, O. Baysal, and M. W. Godfrey, “Code review quality: How developers see it,” in ICSE, 2016, pp. 1028–1038.

[3] A. J. Ko, R. DeLine, and G. Venolia, “Information needs in collocated software development teams,” in ICSE, 2007, pp. 344–353.

[4] A. Mockus and J. D. Herbsleb, “Expertise browser: A quantitative approach to identifying expertise,” in ICSE, 2002, pp. 503–512.

[5] S. Negara, M. Vakilian, N. Chen, R. E. Johnson, and D. Dig, “Is it dangerous to use version control histories to study source code evolution?” in ECOOP, 2012, pp. 79–103.

[6] M. Codoban, S. S. Ragavan, D. Dig, and B. Bailey, “Software history under the lens: A study on why and how developers examine it,” in ICSME, 2015, pp. 1–10.

[7] Y. Higo, S. Hayashi, and S. Kusumoto, “On tracking Java methods with Git mechanisms,” Journal of Systems and Software, vol. 165, p. 110571, 2020.

[8] F. Grund, S. Chowdhury, N. C. Bradley, B. Hall, and R. Holmes, “CodeShovel: Constructing method-level source code histories,” in ICSE, 2021.

[9] A. E. Hassan and R. C. Holt, “C-REX: An evolutionary code extractor for C,” 2004.

[10] T. Zimmermann, “Fine-grained processing of CVS archives with APFEL,” Eclipse Technology eXchange Workshop, 2006.

[11] Q. Tu and M. W. Godfrey, “An integrated approach for studying architectural evolution,” IWPC, 2002, pp. 127–136.

[12] Y. Li, C. Zhu, J. Rubin, and M. Chechik, “Semantic slicing of software version histories,” TSE, vol. 44, no. 2, pp. 182–201, 2018.

[13] pry-git: “A plugin for interactive Git access in Ruby’s REPL,” GitHub, 2020.

[14] K. Huang et al., “ClDiff: Generating concise linked code differences,” ASE, 2018, pp. 679–690.

[15] J.-R. Falleri et al., “Fine-grained and accurate source code differencing,” ASE, 2014, pp. 313–324

[16] Eclipse JGit. https://www.eclipse.org/jgit/

[17] Eclipse JDT (Java Development Tools). https://www.eclipse.org/jdt/

[18] Dig, D., & Johnson, R. (2005). How do APIs evolve? A study of the JDK API. Proceedings of the 21st IEEE International Conference on Software Maintenance (ICSM'05), pp. 83-92.

[19] Raemaekers, S., van Deursen, A., & Visser, J. (2014). Semantic Versioning and Impact of Breaking Changes in the Maven Repository. Journal of Systems and Software, 129, 140-158.

[20] Bavota, G., Canfora, G., Di Penta, M., Oliveto, R., & Panichella, S. (2013). How the Evolution of Source Code Implies the Evolution of Comments: An Empirical Study. Proceedings of the 29th IEEE International Conference on Software Maintenance (ICSM), pp. 190-199.

[21] Hora, A., Robbes, R., & Valente, M. T. (2015). When Should We Refactor Software? Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 299-309.

[21] K. Herzig, S. Just, and A. Zeller, "The Impact of Tangled Code Changes," Proceedings of the 10th Working Conference on Mining Software Repositories (MSR 2013), pp. 121–130, 2013. DOI: [10.1109/MSR.2013.6624024].

[22] B. E. Schaefer, M. P. Robillard, and E. Bodden, "What Stops Developers from Using Static Analysis Tools?", IEEE Transactions on Software Engineering, vol. 45, no. 12, pp. 1233–1256, 2019. DOI: [10.1109/TSE.2018.2818703].

[23] C. Bird, P. C. Rigby, E. T. Barr, D. J. Hamilton, D. M. German, and P. Devanbu, "The Promises and Perils of Mining

Git," Proceedings of the 6th IEEE International Working Conference on Mining Software Repositories (MSR 2009), pp. 1–10, 2009. DOI: [10.1109/MSR.2009.5069475].
[24] Gousios, G., Pinzger, M., & Deursen, A. V. (2014). An Exploratory Study of the Pull-Based Software Development Model. In Proceedings of ICSE, pp. 345–355.
[25] McIntosh, S., Adams, B., & Hassan, A. E. (2015). The Impact of Code Review Coverage and Code Review Participation on Software Quality. In Proceedings of the IEEE/ACM International Conference on Mining Software Repositories (MSR), pp. 192–202.
[26] Mondal, S., Monden, A., & Matsumoto, K. (2022). Empirical Study on the Relation between Refactoring and Code Changes using Version Control Systems. In Journal of Systems and Software, 184, 111161.
[27] Dey, T., Mockus, A. (2020). Incremental Development and the Impact on Software Quality: An Empirical Analysis of GitHub Projects. In IEEE Transactions on Software Engineering, 46(10), pp. 1007–1024.
[28] Bissyandé, T. F., Lo, D., & Klein, J. (2022). Mining Software Repositories: State of the Art and Challenges. In IEEE Software, 39(2), pp. 78–85.
[29] AlOmar, E., AlOmar, K., & Alrumaih, A. (2021). An Empirical Study on Software Change Patterns Using Version Histories. In Information and Software Technology, 134, 106553.
[30] Shang, W., Hassan, A. E., & Adams, B. (2023). Automated Software Evolution Analysis: Recent Advances and Future Directions. In ACM Computing Surveys, 55(1), Article 13.